ETHRC

Dataset & Training Strategy for FPV Drone Racing Gate Detection

A research brief for the ETHRC Aeronautics gate-detection model targeting A2RL and The AI Grand Prix, 2026 season.

Version 0.1 | May 9, 2026 | ETHRC Internal

Aeronautics | Perception working group

Mahbod Tajdini | mtajdini@ethrobotics.ch

TL;DR

The current ETHRC gate detector, after the multi-task keypoint head landed under plan P1-1, reaches **85.1 % detection rate at 1.01 px median corner error on the TII piloted split** (`flight-01p-ellipse`, 423 frames, single forward pass, no GT prior, no EKF state) [1].[1] On separately recorded real-world footage, however, detection still collapses to 39 % (the domain-gap line, unchanged by the keypoint head and tracked under P1-2). The 2026 season targets two competitions whose gates are not represented in our training data: **A2RL** (1.5 m square gates, indoor 100×30 m hall, no team-supplied dataset) [2] and **The AI Grand Prix** (Neros standardised hardware, simulator-driven qualifier, finals in Ohio Nov 2026) [3]. Closing the gap demands more than retraining on more TII data.

Decision	Recommendation
Primary training data	Stay on TII v3.0.0 [1] (already wired).
Supplementary synthetic	Render A2RL-spec 1.5 m gates in the AirSim NeurIPS-2019 environments [4]; layer onto the existing Places365 background pool.
Held-out evaluation	Three-tier: (i) TII val/test, (ii) hand-labelled team footage, (iii) on-site smoke test against a frozen pre-deployment checkpoint.
Fine-tune trigger	When team footage is available; freeze encoder, train decoder + last 2 conv blocks; gate on "no regression on TII val."
Competition-data collection	5–10 min FPV footage of the actual competition gate from ≥3 angles and 2 lighting conditions before each event.

[1] Numbers from `docs/report/07_keypoint_head/findings.md` (`model_direct` mode). The previously-reported 82 % / 5.0 px headline measured QuAdGate refinement *given the GT corners as the prior*: a benchmarking convenience, not a deployment metric. `docs/report/06_realistic_priors/findings.md` documents the prior-sensitivity that made the older number misleading.

1 Problem statement

The repository's gate detector is now a multi-task DeepLabV3+ that emits both a binary ring mask and a 4-channel keypoint heatmap (one channel per inner gate corner) in a single forward pass [1]. On the held-out TII piloted split, the keypoint head's corners alone (no QuAdGate refinement, no prior) deliver ∼85 % detection at 1.01 px median corner error [1].[2] On a separately recorded real-world flight using the same drone hardware, however, detection still collapses to 39 % (the README's standing domain-gap entry). The README lists the symptoms: the model "segments non-gate regions as gates," degrading both detection and downstream PnP.[3]

This is the textbook *domain shift* problem in drone-racing perception: training data is dominated by one environment and one gate appearance, and the network specialises rather than generalises. [5] document the same effect under nothing more than illumination change in a controlled drone arena, and propose a continual-learning rehearsal scheme to mitigate forgetting across lighting conditions.

A2RL and The AI Grand Prix shift the domain twice: first through different gate geometry and material (1.5 m powder-coated frames at A2RL; not-yet-published gates at AIGP), then through different camera intrinsics (rolling-shutter 820×616 monocular at A2RL [2]; ∼12 MP wide-angle on the Neros airframe at AIGP [3]). Without a deliberate plan, the 39 %-detection regime is the most plausible outcome at competition.

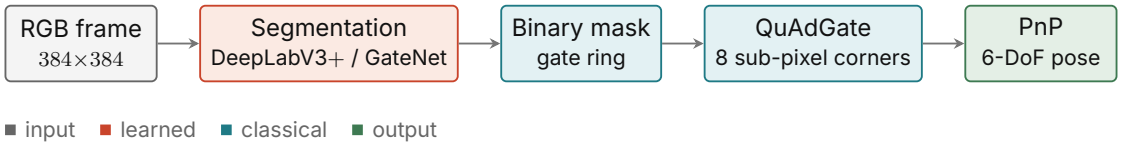


Figure 1. The two-stage gate-detection pipeline used in this repo. The learned segmentation backbone runs once per frame; QuAdGate then extracts sub-pixel inner and outer corners by intersecting LSD line segments and matching them against fixed prior bit patterns (`src/gate_detection/postprocessing/quadgate.py:32--35`). The structure mirrors the A2RL-winning MonoRace pipeline (Fig. 5 of [2]).

2 Competition specifications

2.1 A2RL: Abu Dhabi Autonomous Racing League

The April 2025 edition was held over 11–12 April 2025 at ADNEC, Abu Dhabi. Authoritative specs are documented by the winning team [2]:

- Track: 11 square gates, 1.5 m inner opening, 2.7 m outer frame, in a 100×30 m hall (track itself $76 \times 18 \times 5.4$ m).
- Drone: 966 g, NVIDIA Jetson Orin NX companion + STM32H743 flight controller; monocular rolling-shutter CMOS at 820×616, FoV $155° \times 115°$; 1 kHz/2 kHz IMU.

[2] Per `docs/report/07_keypoint_head/findings.md` (`model_direct` mode on `flight-01p-ellipse`). The historical ∼82 % / 5.0 px numbers measured QuAdGate refinement under a perfect-prior benchmarking convenience and have been re-cast as historical baselines, not deployment metrics; see `docs/report/06_realistic_priors/findings.md` for the prior-sensitivity ablation.

[3] `README.md` §Known issues.

- Provided to teams: *none*. No pre-recorded dataset and no official simulator. Teams developed synthetic data and gym environments themselves [2].
- Best result reported: peak 28.23 m/s (≈ 100 km/h), fastest lap 16.56 s.
- 2026 edition: announced; $1 M prize pool [6].

The lack of an official dataset is the most consequential fact in this section. It implies that *any* team's training material has to be either (a) collected on-site during the limited practice days, or (b) synthesised against the published gate geometry. Both paths are expected; we should plan for both.

2.2 The AI Grand Prix

Per the official site [3], the 2026 season is structured as:

- **Virtual qualifier 1**: May to early July, sparse gates, desaturated environment, gates visually highlighted.
- **Virtual qualifier 2**: late July, real 3D-scanned environment, more gates.
- **Physical qualifier**: September 2026, Southern California, two-week training window. Indoor, "consistent lighting"; no wind.
- **Finals**: November 2026, Ohio. Indoor; possible additional disturbances from spectator camera flashes.
- Hardware: Neros standardised drone, ∼12 MP wide-angle FPV monocular, ∼100 TOPS onboard compute, no LiDAR, no hardware modifications allowed.
- Tooling: simulator package released May 2026, Python interface specified late March, weekly updates.
- Prize: $500 K pool; Anduril job offer for one finalist.

Compared with A2RL, AIGP *does* provide a simulator and a flight-data export pipeline, so the qualifier path can be executed entirely from synthetic data. The catch: between Round 1 and Round 2, environmental complexity rises sharply: the perception stack must survive the move from a desaturated arena to a 3D-scanned real environment, and again at the physical qualifier where the simulator parity is unknown.

3 Public-dataset survey

The table below lists every public, drone-racing-adjacent dataset we evaluated. Verdicts in the right column reflect compatibility with our existing pipeline (segmentation → inner-corner keypoints → PnP), *not* dataset quality in the abstract.

Dataset	Year	Real / Sim	Labels	Verdict
TII v3.0.0 [1]	2025	Real	17-col YOLO-keypoint, 4 inner corners + visibility, 36 flights, CC BY 4.0	**Primary.** Already wired into our `discovery.py`.
UZH-FPV [7]	2019	Real	6-DoF pose ground truth (Leica), no gate labels	Useful for VIO regression tests; no direct value for gate detection unless we re-annotate.
Blackbird [8]	2018	Real	Camera + motor telemetry, ∼4.9 TB; no gate labels	Out of scope: aggressive-flight perception, no gates.
AirSim DRL [4]	2019	Sim	Programmatic gate poses; render any annotation	**Use as a synthesis harness**, not as a fixed dataset. Repository archived, but the simulator still runs.
GateNet AU-DR [9]	2021	Real + Sim	Gate *pose* (centre/distance/orientation) on fish-eye images	Single-pose label, not corners; secondary. Useful for cross-validating our PnP outputs.
PencilNet synth [10]	2022	Sim	31 k UE4 fish-eye renders, gate pose tuple	Single-pose label. Useful as a sim-to-real reference, not as direct training data.
AlphaPilot 2019	2019	Real	28 k images, bbox + corner keypoints	**Closest in spirit to our label format, but *not publicly distributed*.** Reported only via challenge writeups; no archival URL we could verify.

Take-away. Only one dataset in this list, TII, delivers labels that fit our exact pipeline without re-annotation, and it is the one we already train on. Every other public source either (a) lacks gate labels, (b) provides single-pose labels rather than per-corner keypoints, or (c) is closed. Closing the competition domain gap therefore cannot be done by adding more public data; it must be done by adding synthetic data (with our own labels) and by collecting and labelling real footage from our own flights and from each competition venue.

4 Gate segmentation method

The dataset choice in §3 only matters once we know what the segmentation *model* is doing with the data. This section specifies (4.1) the supervision target, (4.2) the two backbones the repository can train, (4.3) what 2024–2026 published methods do instead, and (4.4) an honest pros / cons read of the current design against that benchmark. Every claim that follows is anchored either to a file:line in the repository or to a paper whose abstract was re-fetched and whose claim was quoted verbatim.

4.1 What ``gate segmentation'' means in this repo

The segmentation head outputs a single-channel **binary ring mask** on a 384×384 RGB input (`configs/default.yaml:5`). The ring is the band between the gate's inner and outer rectangles; only the four *inner* corners are labelled in the TII dataset, so outer corners are synthesised offline by radial scaling about the inner centroid with factor 1.4 (`benchmark_pnp.py:79--81`, exactly outer $=$ centroid $+\,1.4\cdot($inner $-$ centroid$)$). The training mask is then "fill outer polygon, erase inner polygon" (`src/gate_detection/data/labels.py:103--146`).

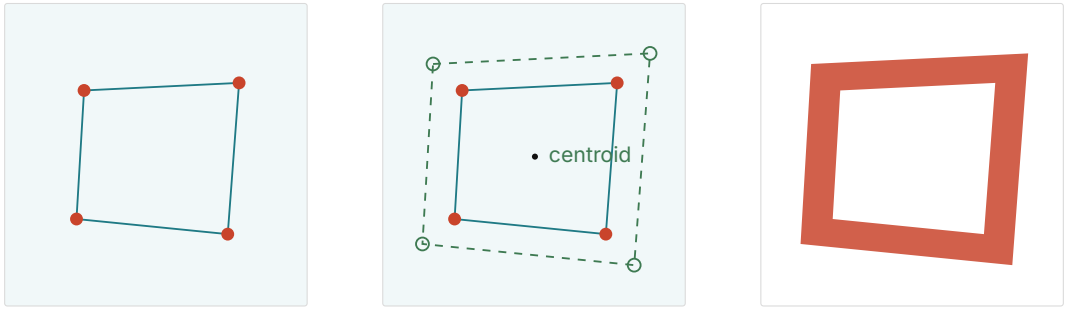


Figure 2. Ring-mask construction. **(a)** The dataset only labels the four *inner* corners of each gate (coral dots). **(b)** The outer corners (forest rings) are synthesised offline by radial scaling about the inner centroid with factor 1.4, i.e. outer $=$ centroid $+\,1.4\cdot($inner $-$ centroid$)$ (`benchmark_pnp.py:79--81`). **(c)** The training target is the binary ring between the two quadrilaterals (`src/gate_detection/data/labels.py:103--146`); the gap is exactly the line-detector signal QuAdGate keys on for sub-pixel corner extraction.

The ring exists for QuAdGate. The downstream corner extractor (`src/gate_detection/postprocessing/quadgate.py`) runs OpenCV's Line Segment Detector on the binary mask, intersects line pairs inside the ring, and classifies each intersection into a corner type (TL/TR/BR/BL) by matching a 4-bit local mask sample against fixed hand-crafted prior bit patterns (`quadgate.py:32--35`). MonoRace, the A2RL April 2025 champion, runs the same overall recipe ([2], Fig. 5–6). A filled quadrilateral target would erase the ring boundary the line-detector keys on; the ring is therefore not an aesthetic choice, it is the right supervision shape for a PnP-driven geometry-first detector.

4.2 What we have today

Two interchangeable backbones are exposed by the repository (`src/gate_detection/models/`):

- **GateNet** (`models/gatenet.py:20--77`): a narrow custom U-Net with channel divisor $f{=}4$, yielding stage widths $[16, 32, 64, 128, 128]$ and four upsampling stages. It emits five sigmoid heads y_0 through y_4 — one at the bottleneck, one at full resolution, three inter-

mediate — that supply deep-supervision losses with weights $(4, 2, 1, 1, 1)$ (`losses/segmentation.py:30--34`).

- **DeepLabV3+** (`models/deeplabv3plus.py:11--37`): `segmentation_models_pytorch` wrapper, ResNet-34 encoder, ASPP atrous-rate dilation, single sigmoid head. The `encoder_weights` flag accepts `"imagenet"` but defaults to `None` in `models/deeplabv3plus.py:14`, and `configs/default.yaml` does not override it, so ResNet-34 currently trains from scratch.

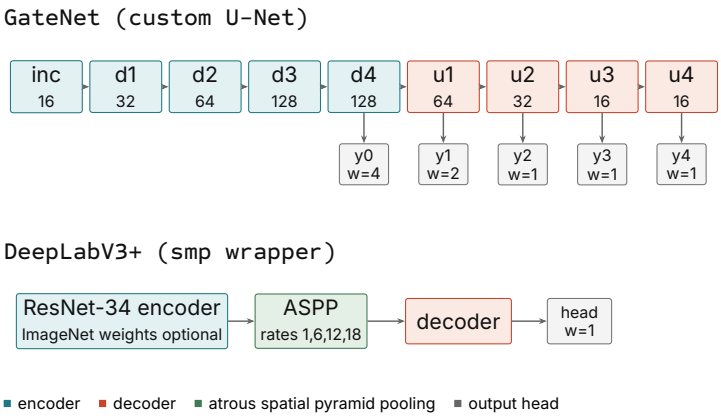


Figure 3. The two interchangeable backbones in `src/gate_detection/models/`. **GateNet** is a narrow U-Net with channel divisor $f{=}4$ (stage widths $[16, 32, 64, 128, 128]$) and five deep-supervision heads (y_0 at the bottleneck, y_4 at full resolution, three intermediate); the per-scale loss is summed with weights $(4, 2, 1, 1, 1)$ and only y_4 is used at deployment. **DeepLabV3+** is single-headed (single-scale Dice $+2\times$BCE) and supports ImageNet-pretrained ResNet-34, but `encoder_weights=None` is the default in `models/deeplabv3plus.py:14`, so ResNet-34 currently trains from scratch. Both backbones feed the same QuAdGate corner extractor.

Loss. Per-scale Dice $+\ 2.0\times$BCE, summed with the deep-supervision weights above (`losses/segmentation.py:30--60`). Dice uses Kornia's binary variant on a two-channel (bg, fg) tensor; BCE is forced to fp32 inside `torch.autocast` to avoid bf16 $\log(0)$ overflow (`losses/segmentation.py:57--58`).

Training. AdamW at $3{\times}10^{-4}$ with weight decay 10^{-4}, polynomial-HF schedule (warmup 0.1, power 0.9, $\text{lr}_{\text{end}}{=}10^{-6}$), batch 8, 100 epochs, bf16 AMP, `torch.compile(mode="max-autotune")`, channels-last memory format (`configs/default.yaml:18--40`).

Augmentation. Online: horizontal flip, $\pm 15°$ affine with ± 10 % translation and 0.9–1.1 scale, HSV jitter ± 10 on hue and 0.8–1.2 on saturation/value, motion blur with kernel 5–15 px at random angle, and Gaussian noise with $\sigma \leq 15$ (`src/gate_detection/data/transforms.py:91--98`). Offline: composite gates onto Places365-custom indoor backgrounds at $5\times$ multiplier via `scripts/augment_data.py` and `scripts/download_backgrounds.py`.

Mask refinement via SAM2. Training masks are not just rasterised polygons; `configs/default.yaml:7` sets `use_sam_masks:true`, and `scripts/generate_sam_masks.py` runs SAM2 (`facebook/sam2.1-hiera-large`, `models/sam2.py:18`) with the labelled bounding box as a prompt, then subtracts the inner polygon to obtain a ring-shaped *soft* mask. This pre-processing happens once; SAM2 is not in the inference path. The technique is uncommon in published gate-detection work and is one of the more interesting bets in the codebase.

4.3 What 2024--2026 SOTA looks like

Three families dominate recent published drone-racing perception:

- **Pure keypoint regression on a YOLO backbone** is what currently wins on real hardware.

The A2RL 2025 podium entry [11] runs YOLOv8-Pose (s) at 640×640, reports 0.971 keypoint mAP and 0.877 bounding-box mAP, and clears 16.1 ms (62 Hz) on a Jetson Orin NX after TensorRT export. AlphaPilot's 2019 challenge runner-up [12] pioneered learned-keypoint gate detection. The nano-drone Crazyflie work of [13] regresses four corner coordinates from 160×160 monochrome on a GAP8 MCU at 1.4 px RMSE / 30 Hz.

- **Pure-mask methods, illumination-first**: PencilNet [10] achieves zero-shot sim-to-real by pre-processing every image with an edge-emphasising morphological filter (Algorithm 1 of that paper); the continual-learning extension [5] uses the same backbone and a DER++ rehearsal buffer to recover from illumination-shift catastrophic forgetting.
- **Hybrid (mask + classical corners)** is what MonoRace, the A2RL April 2025 champion, runs ([2], Fig. 5–6): a learned segmentation backbone, then QuAdGate corner extraction by LSD line intersection, then PnP $\rightarrow$ EKF $\rightarrow$ controller. This is the *same* pipeline our repository runs.

Mask only	Keypoint only	Hybrid (mask + corners)
PencilNet [10] sim-only $\rightarrow$ pencil-filtered fish-eye; 32 k params; TX2	**AlphaPilot** [12] 2019 challenge runner-up; learned keypoints	**MonoRace** [2] A2RL 2025 winner; segmentation + QuAdGate corners
GateNet 2021 [9] fish-eye gate *pose*; 60 Hz on TX2	**YOLOv8-Pose** [11] A2RL 2025 podium; 0.971 kp mAP, 62 Hz Orin NX	**[• This work]** ring-mask + QuAdGate + multi-task keypoint head; SAM2 label refinement
Continual-learning [5] PencilNet backbone + DER++ rehearsal	**Nano-drone PULP-FCN** [13] 4 corners; 1.4 px RMSE, 30 Hz GAP8	**85 % TII / 1 px median** (no prior); 39 % real (domain gap, open)

Figure 4. Where this brief sits in the 2024–2026 method landscape. Pure-mask methods (teal) optimise an illumination-robust input representation but rely on classical post-processing for pose. Pure-keypoint methods (coral) regress 4–8 corner coordinates directly and are the current podium-winning approach on real A2RL hardware. Hybrid methods (forest) keep both — a segmentation mask for descriptor matching and corner keypoints for PnP. This repository sits in the hybrid column; the keypoint column is what is currently winning in raw real-world numbers.

Two adjacent threads worth knowing: **NeRF-rendered training environments** (FalconGym [14] reports 95.8 % real-flight success and 10 cm crossing error on 38 cm-radius gates after training only in NeRF), and **foundation-model distillation** (MobileSAM [15] runs at $\sim$10 ms with $5\times$ fewer parameters than FastSAM and $60\times$ fewer than SAM ViT-H, but no published drone-racing application). On loss design, the segmentation-loss survey of [16] catalogues 25 variants of binary-segmentation losses, with Dice and cross-entropy among the canonical building blocks — the combo we already use.

4.4 Is what we have good?

Yes, on paper. No, on real footage. The architecture and loss choices are well-aligned with peer-reviewed practice. The failure mode is generalisation, not method.

- **Hybrid target representation.** Ring-mask + classical corner extraction is the same pipeline shape as the A2RL champion's [2]; geometrically it is the right place to stand for PnP-based pose recovery.
- **Loss combo.** Dice + $2\times$BCE pairs two canonical segmentation losses catalogued in the survey of [16]; the Dice term anchors region overlap, the BCE term keeps

gradients well-behaved on the long ring boundary.

- **Deep supervision** on the GateNet variant is consistent with current evidence in segmentation literature (the bottleneck head supplies a stable gradient when full-resolution heads start collapsing).
- **SAM2 as a label refiner** (`configs/default.yaml:7`, `scripts/generate_sam_masks.py`) is an underexplored angle — no comparable published drone-racing work uses SAM2 this way. Potentially differentiating.

- **ImageNet weights are off by default.** `models/deeplabv3plus.py:14` sets `encoder_weights=None` and `configs/default.yaml` does not override it, so ResNet-34 trains from scratch. Turning ImageNet weights on is one of the cheapest accuracy levers available for a binary segmentation backbone.
- **No ImageNet normalisation.** Pre-processing is BGR→RGB and `/255` only (`src/gate_detection/inference/predictor.py:175--177`); per-channel mean/std subtraction is not applied. If we ever turn ImageNet weights on, this will silently undermine them.
- **No keypoint auxiliary head.** The published podium entry [11] trains keypoints jointly with the bounding box at 0.971 mAP. Our model regresses corners only via QuAdGate at inference, so we miss a strong auxiliary signal during training.
- **No PencilNet-style edge augmentation, no NeRF synthesis.** The two interventions with the strongest 2022–2025 real-world transfer evidence [10, 14] are both absent. The 39 % on real footage is consistent with that omission.

1. Set `encoder_weights:imagenet` in `configs/default.yaml`, and add ImageNet mean/std normalisation in the predictor. *Cheap, large expected lift.*
2. Add a parallel keypoint-heatmap head (4 inner corners as Gaussian peaks); train multi-task with a small weight on the existing mask loss. Pattern from [11].
3. Bake an edge-emphasising channel (PencilNet's morphological filter, [10] Alg. 1) into the offline augmentation pipeline as a fourth input channel; the same filter has subsequent real-flight illumination-robustness evidence in the continual-learning extension of [5].
4. Distil SAM2's mask outputs into a small student backbone (e.g. MobileSAM-style, [15]) and benchmark vs hand-labelled ring masks. Novel contribution if it works; clean fallback if it does not.

5 Recommended training stack

Primary corpus. TII v3.0.0 [1], already wired through `src/gate_detection/data/discovery.py` (28 train / 8 val flights, deterministic split with `random.seed(42)`).

Synthetic supplement. Render A2RL-spec 1.5 m / 2.7 m square gates in the AirSim NeurIPS-2019 environments [4] via the existing object-pose API, scripting random gate placement / pose / scale and reading back ground-truth corners through `simSetSegmenta`

`tionObjectID`. The repository's offline augmentation pipeline (`data/synthesis.py`) already composites gates onto Places365 backgrounds; the AirSim renders extend the same logic to physically plausible camera poses.

Background pool. The existing `scripts/download_backgrounds.py` pull from the Places365-custom set on Hugging Face is sufficient for general indoor diversity. Before each competition we should additionally photograph 30–50 venue-style background frames (no gates) to push the offline augmenter toward the actual lighting and clutter we will see.

Loss & head. No change. Deep-supervised Dice + BCE on the segmentation backbone, QuAdGate corner extraction post-hoc; the documented architecture in `docs/architecture.md` stays.

6 Testing tactic

Failure mode to design against: a checkpoint that reports 82 % on the TII test split but collapses on real footage. The current evaluation reports a single TII number and is therefore blind to that collapse until the drone is in the air. The fix is to run three independent evaluations on every candidate checkpoint, in this order:

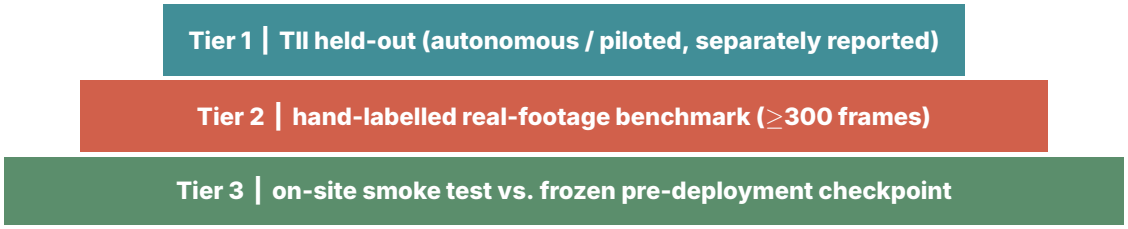


Figure 5. Three-tier evaluation. Each tier catches failure modes the cheaper one cannot: TII held-out catches segmentation regressions; the real-footage benchmark catches the domain shift the brief is built around; the on-site smoke test catches venue-specific surprises. A checkpoint advances only after all three pass.

6.1 Tier 1: TII held-out, split by mode

Existing `scripts/evaluate.py` on the TII val/test set, but *report autonomous and piloted IoU separately*. The two flight modes have different gate distances, motion blur regimes and label noise; pooling them masks regressions on the harder mode.

Pass criterion: no autonomous-only or piloted-only IoU drop > 2 pp from the previous accepted checkpoint.

6.2 Tier 2: Real-footage benchmark (to be built)

Hand-label ∼300 frames from team-recorded flights using `scripts/viewer.py`; the "real" eval set must include lighting conditions, gate distances and gate orientations *not* represented in TII. Track detection rate, IoU and reprojection error separately.

Pass criterion: real-footage detection rate ≥60 % before any field deployment; reprojection error within 2× of the TII number. Anything weaker means we are still in the regime where the README documents 39 %, and we should not believe the simulator either.

6.3 Tier 3: On-site smoke test

At each competition, the first practice slot is a *regression check*: run inference on the venue feed for $\geq$3 minutes, store the raw frames + masks, and compare against a frozen pre-deployment checkpoint. Any new false-positive class (e.g. a wall structure being segmented as gate) is an instant abort to the previous checkpoint.

Pass criterion: no novel false-positive class beyond the offline real-footage benchmark.

7 Fine-tuning on competition data

Should we fine-tune? Yes, but only with real samples from the actual competition gate, and only after we have a labelled real-footage benchmark to detect regression.

Why "yes". [5] demonstrate that under nothing more than illumination change, gate-detection performance degrades enough to require explicit adaptation, and that a small rehearsal buffer recovers most of it. [10] show that synthetic-only training can be made to transfer zero-shot *only* via a domain-invariant filter (their "pencil" edge representation); without such a trick, sim-to-real falls off. The MonoRace team, the only team to have actually solved the A2RL course at champion level, relied on synthetic data plus an offline self-supervised geometric calibration step keyed off known gate dimensions [2]. Their pipeline is the strongest single existence proof that competition-time adaptation is needed.

7.1 What to collect

For each competition, before the first scored run:

- 5–10 minutes of FPV footage of the actual competition gate, taken from $\geq$3 spatial angles (head-on, 30–45°, oblique) and $\geq$2 lighting conditions (with *and* without venue stage lights).
- Static photographs of the gate corners with a tape measure visible, to confirm published dimensions.
- Background frames with no gates, for the offline augmenter (see §4).

7.2 How to label

1. Run the current best checkpoint on every collected frame to produce *pseudo-labels*.
2. Open `scripts/viewer.py` (already present); accept / reject / edit corners by hand.
3. Stop when a Tier-2-shaped 200–300 frame split exists.

7.3 What to update, what to freeze

- Freeze the encoder (ResNet-34 or equivalent in our backbone) for the first stage of fine-tuning.
- Train the decoder and the last two conv blocks at $0.1\times$ the original learning rate.
- Co-train with a *rehearsal mini-batch* from TII (1:1 ratio at minimum) to prevent catastrophic forgetting [5].
- Stop early: "best on competition split, no regression on TII val" is the joint criterion. If the joint criterion cannot be satisfied within 10 epochs, the dataset is too small or too biased; collect more, do not train longer.

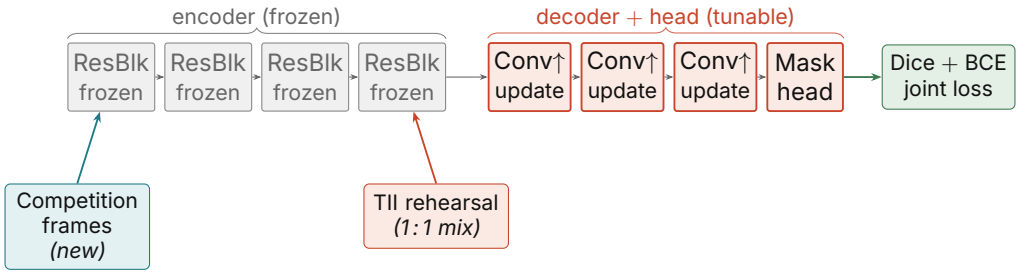


Figure 6. Fine-tuning protocol. Encoder weights are frozen; only the decoder and the mask head update, at $0.1\times$ the original learning rate. Each batch mixes newly-collected competition frames with a TII rehearsal sample ($1:1$) — an explicit guard against the catastrophic-forgetting failure mode [5] reports for naïve sequential fine-tuning under shifting illumination.

7.4 Validation gate before deployment

1. **TII val** — no >2 pp IoU regression on either autonomous-only or piloted-only.
2. **Real-footage benchmark** — detection rate ≥ 60 %, reprojection error within $2\times$ the TII number.
3. **On-site smoke test** — no novel false-positive class beyond the offline benchmark.

All three must pass. A regression on *any* tier is an automatic abort to the previous checkpoint.

8 Risks & open questions

The **45.6° rotation error vs. VICON** reported in `README.md:38--42` is too large to be a detector failure alone — it strongly suggests a coordinate-frame sign or extrinsics bug in the PnP stage. Even a perfect detector cannot recover that. Treat it as an independent workstream that gates everything downstream.

- **A2RL gate appearance unconfirmed.** We have geometry from [2], but colour, surface finish, and reflectivity are not pinned. Plan for a worst-case white powder-coated frame against grey concrete: the failure mode where every wall edge looks gate-like. Confirm against ASPIRE technical bulletins as soon as 2026 specs are published.
- **AIGP camera intrinsics.** The Neros camera differs from the TII rig; we should ingest a Round-1 simulator frame as soon as the May 2026 release lands and re-run undistortion against our intrinsics file.
- **No public AlphaPilot dataset** despite multiple writeups citing one. Our plan must not depend on access; the synthesis pipeline plus on-site collection has to be sufficient on its own.
- **Compute budget for synthetic rendering.** The AirSim repository is archived and un-

maintained; if it breaks under macOS / current Unreal builds, falling back to a NeRF-based render of an A2RL-spec gate is a reasonable plan B but requires effort not yet scoped.

9 References

[1] TII Drone Racing Group. *Race Against the Machine: A Fully-Annotated, Open-Design Dataset of Autonomous and Piloted High-Speed Flight (v3.0.0)*. 36 flights, CC BY 4.0. Oct. 2025. url: https://github.com/tii-racing/drone-racing-dataset.

[2] Stavrow A. Bahnam et al. "MonoRace: Winning Champion-Level Drone Racing with Robust Monocular AI". In: *arXiv preprint arXiv:2601.15222* (2026). Winner of the A2RL drone race, Abu Dhabi, 11–12 April 2025. url: https://arxiv.org/abs/2601.15222.

[3] The AI Grand Prix. *The AI Grand Prix*. 2026. url: https://www.theaigrandprix.com/.

[4] Microsoft Research. *Game of Drones: A NeurIPS 2019 Competition*. AirSim-based simulator; archived, no longer maintained. 2019. url: https://github.com/microsoft/AirSim-NeurIPS2019-Drone-Racing.

[5] Zhongzheng Qiao et al. "Continual Learning for Robust Gate Detection under Dynamic Lighting in Autonomous Drone Racing". In: *International Joint Conference on Neural Networks (IJCNN)*. 2024. url: https://arxiv.org/abs/2405.01054.

[6] ASPIRE. *A2RL: Abu Dhabi Autonomous Racing League*. 2026. url: https://a2rl.io/.

[7] Jeffrey Delmerico et al. "Are We Ready for Autonomous Drone Racing? The UZH-FPV Drone Racing Dataset". In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2019. url: https://fpv.ifi.uzh.ch/.

[8] Amado Antonini et al. "The Blackbird Dataset: A Large-Scale Dataset for UAV Perception in Aggressive Flight". In: *International Journal of Robotics Research* (2020). url: https://github.com/mit-fast/Blackbird-Dataset.

[9] Huy Xuan Pham et al. "GateNet: An Efficient Deep Neural Network Architecture for Gate Perception Using Fish-Eye Camera in Autonomous Drone Racing". In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2021. url: https://github.com/open-airlab/GateNet.

[10] Huy Xuan Pham, Andriy Sarabakha, Mykola Odnoshyvkin, and Erdal Kayacan. "PencilNet: Zero-Shot Sim-to-Real Transfer Learning for Robust Gate Perception in Autonomous Drone Racing". In: *arXiv preprint arXiv:2207.14131* (2022). url: https://arxiv.org/abs/2207.14131.

[11] Maulana Bisyir Azhari et al. "Drift-Corrected Monocular VIO and Perception-Aware Planning for Autonomous Drone Racing". In: *arXiv preprint arXiv:2512.20475* (2025). A2RL 2025 podium entry; YOLOv8-Pose at 640×640, 0.971 keypoint mAP / 0.877 bbox mAP, 62 Hz on Jetson Orin NX (TensorRT 8.5.2). url: https://arxiv.org/abs/2512.20475.

[12] Philipp Foehn et al. "AlphaPilot: Autonomous Drone Racing". In: *Robotics: Science and Systems (RSS)*. 2020. url: https://arxiv.org/abs/2005.12813.

[13] Lorenzo Scarciglia, Antonio Paolillo, and Daniele Palossi. "A Map-free Deep Learning-based Framework for Gate-to-Gate Monocular Visual Navigation aboard Miniaturized Aerial Vehicles". In: *arXiv preprint arXiv:2503.05251* (2025). Crazyflie 2.1 + GAP8; 160×160 monochrome input; 1.4 px corner RMSE, 30 Hz closed-loop control. url: https://arxiv.org/abs/2503.05251.

[14] Yan Miao, Will Shen, and Sayan Mitra. "FalconGym: A Photorealistic Simulation Framework for Zero-Shot Sim-to-Real Vision-Based Quadrotor Navigation". In: *arXiv preprint arXiv:2503.02198* (2025). NeRF-rendered training environment; 95.8 % real-flight success on 38 cm-radius gates with 10 cm crossing error. url: https://arxiv.org/abs/2503.02198.

[15] Chaoning Zhang et al. "Faster Segment Anything: Towards Lightweight SAM for Mobile Applications". In: *arXiv preprint arXiv:2306.14289* (2023). MobileSAM: ∼10 ms per image (8 ms encoder, 4 ms decoder); 5× faster than FastSAM, 60× smaller than SAM ViT-H. url: https://arxiv.org/abs/2306.14289.

[16] Reza Azad, Moein Heidary, Kadir Yilmaz, et al. "Loss Functions in the Era of Semantic Segmentation: A Survey and Outlook". In: *arXiv preprint arXiv:2312.05391* (2023). Comprehensive review of 25 segmentation losses, including Dice, BCE, focal, and combined variants. url: https://arxiv.org/abs/2312.05391.